

Computer Use Agents (CUA)

Task:

Please navigate to a new Chrome window, navigate to claude.ai, and once loaded, ask Claude to create a personal website with a 90s style theme.

CUA Agent:

Planning. Reasons and creates a plan on how to carry out the task. This step requires a general knowledge about the environment workspace and the user interface.

Grounding. Carries out the cursor and keyboard-related works, such as clicking, typing, moving the cursor, etc. This step requires one-to-one mapping between the UI elements and their coordinates on the screen.

Goal:

To collect a large-scale Grounding dataset from openly available YouTube tutorials with no human annotations.

Step 1



The plan

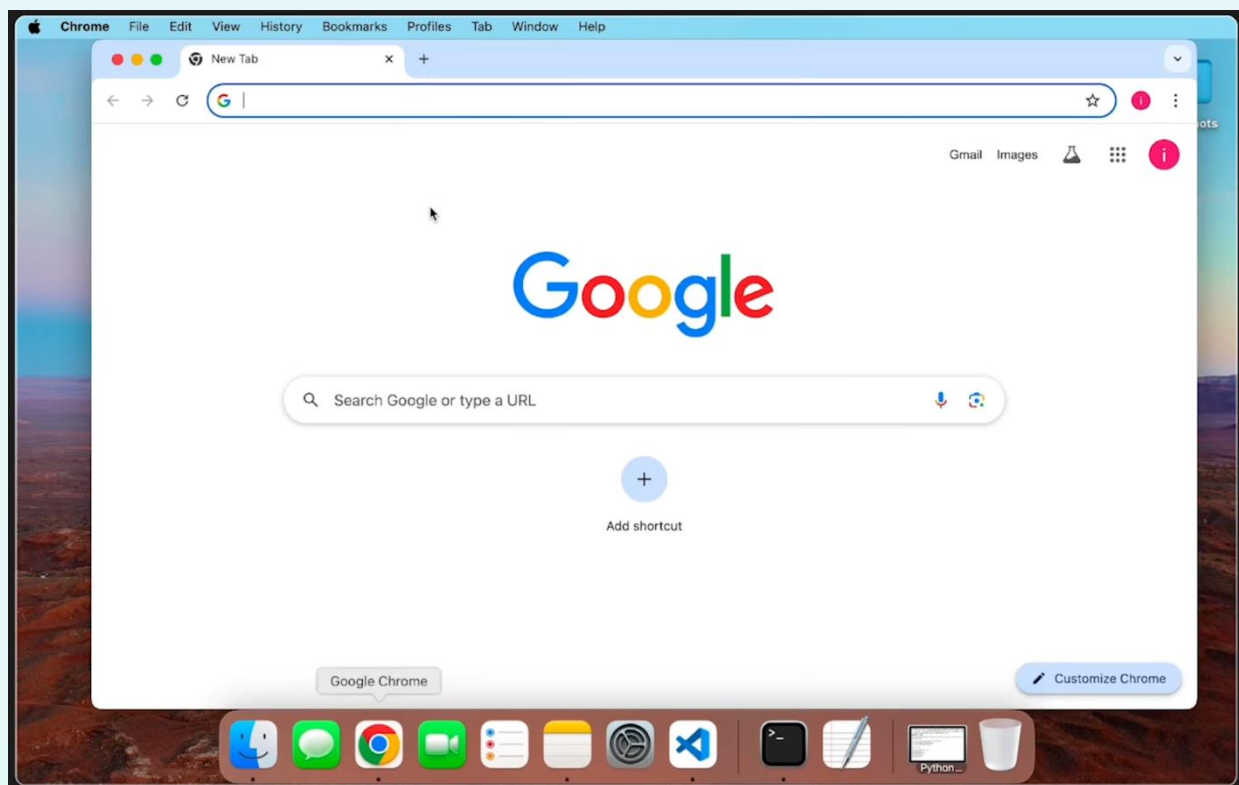
I'll help you navigate to claude.ai and interact with it. Let me break this down into steps:

1. First, let's open Chrome
2. Then we'll go to claude.ai
3. Once loaded, we'll type the request
4. Finally, submit the request

Let's start:

Move to [334, 724]
Left click

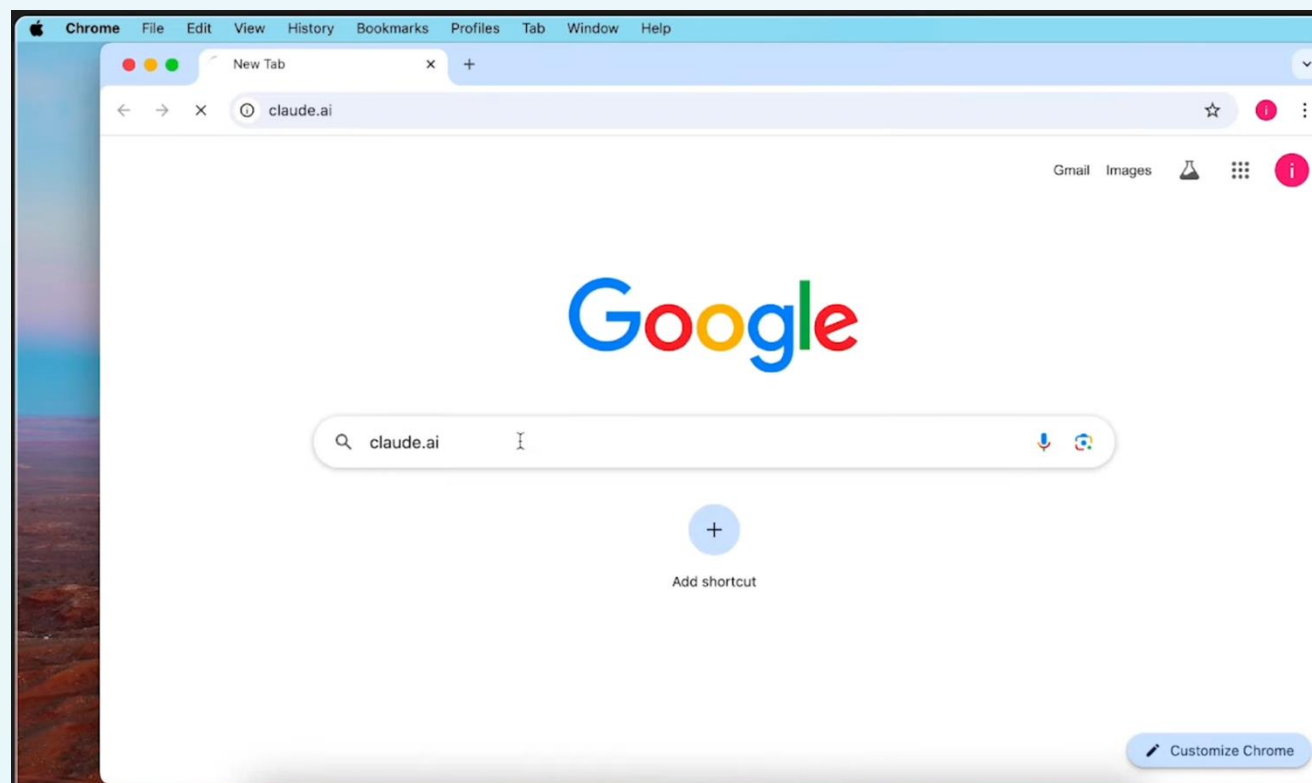
Step 2



Now, I'll type in claude.ai in the Google search bar and navigate to the website

Move to [525, 382]
Left click
Type [claude.ai]
Key [Enter]

Step 3



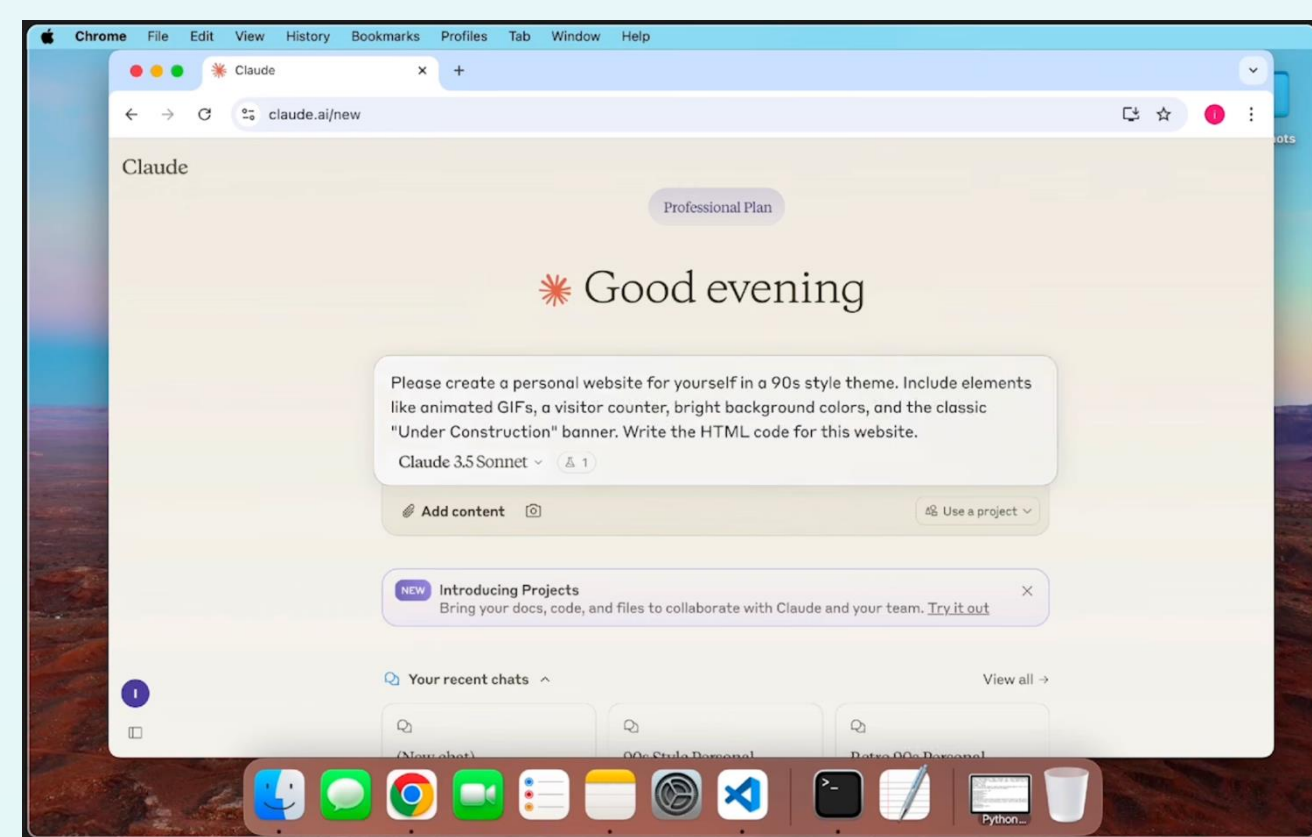
I notice we've successfully reached claude.ai and the interface is ready for input. Now I'll ask Claude to create a 90s style personal website. First, I'll need to click in the input area and type the request.

Move to [500, 450]
Left click
Type [claude.ai]

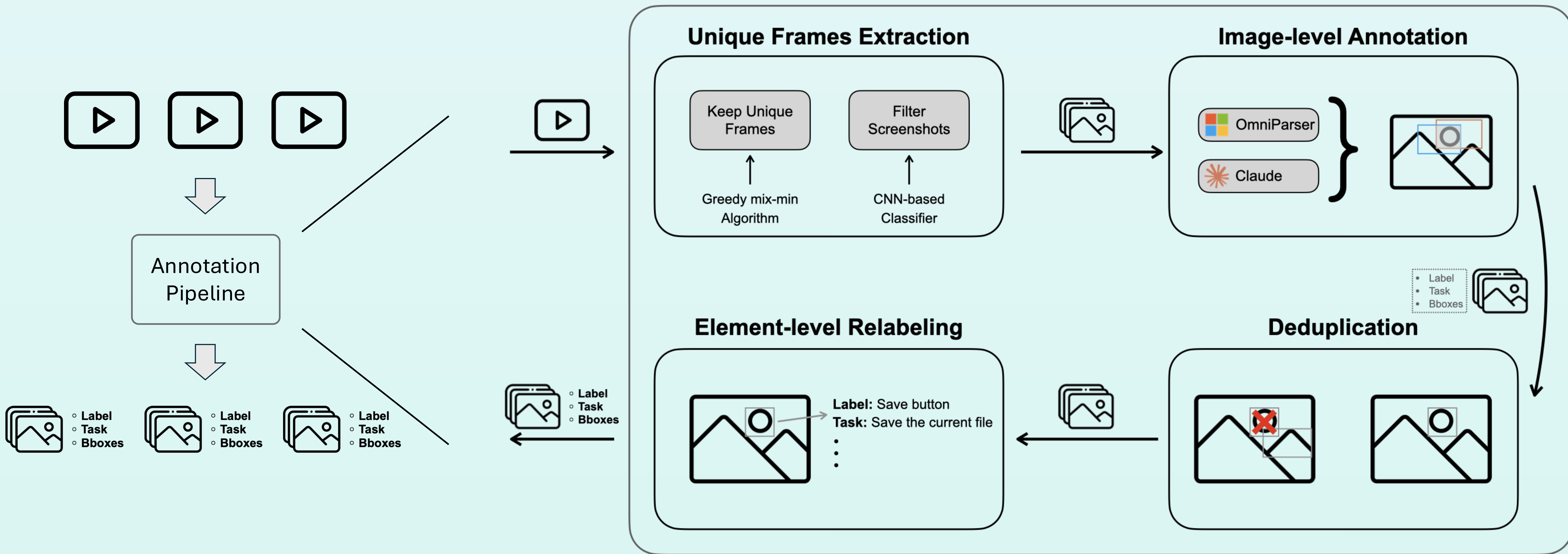
"Please create a personal website for yourself in a 90s style theme. Include elements like animated GIFs, a visitor counter, bright background colors, and the classic 'Under Construction' banner. Write the HTML code for this website."

Key [Enter]

Step 4



Annotation Pipeline



Our annotation pipeline takes openly available YouTube video as input and outputs a list of unique screenshots from the video with unique UI elements annotated with:

- **Label:** the UI element's label
- **Bboxes:** bounding boxes coordinates
- **Task:** a possible task that can be done using that UI element.

The annotation pipeline consists of 4 steps: (1) extraction of unique frames from a video, (2) image-level annotation of UI elements, (3) deduplication of UI elements, (4) element-level annotation of UI elements.

1. **Unique Frames Extraction:** A video goes through a min-max algorithm using image-hashing to keep only unique frames, which are then passed to a CNN-based classifier to filter out non-screenshot images.
2. **Image-Level Annotation:** The UI elements in an image are detected using OmniParser and Claude-sonnet-4.5. We then take the overlapping bounding boxes as the final output.
3. **Deduplication:** Deduplication is carried out using an OCR text and the coordinates of the UI elements within an image. OCR text is detected using olmOCR-7B model.
4. **Element-level Relabeling:** We use Gemini-3-flash to relabel individual UI elements (optional step).

Analysis of the Dataset

Dataset:

We ran our pipeline on 10 tutorial videos on **Power Point**, which generated:

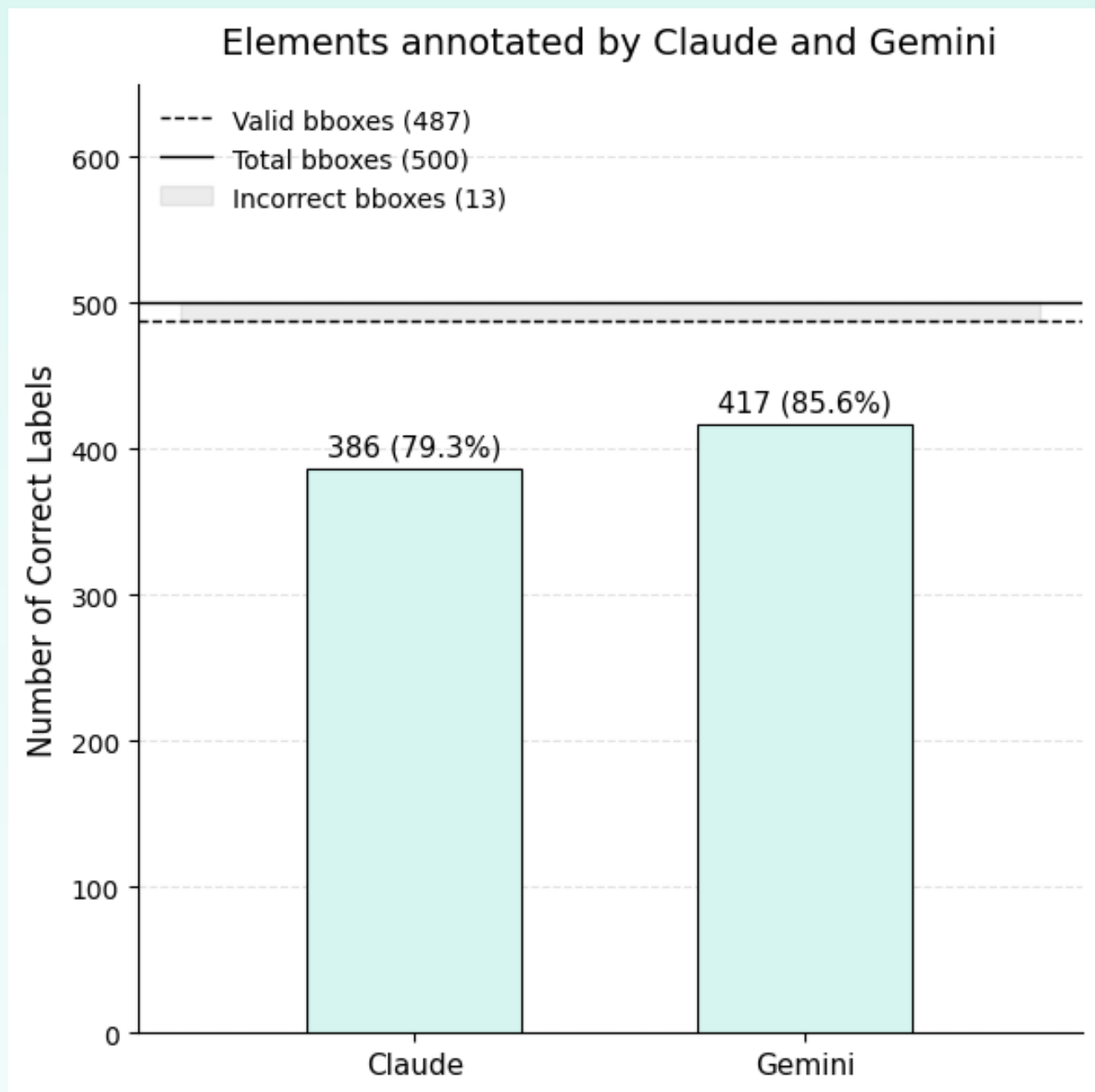
- 585 unique screenshots
- 13,492 unique UI elements

Evaluation:

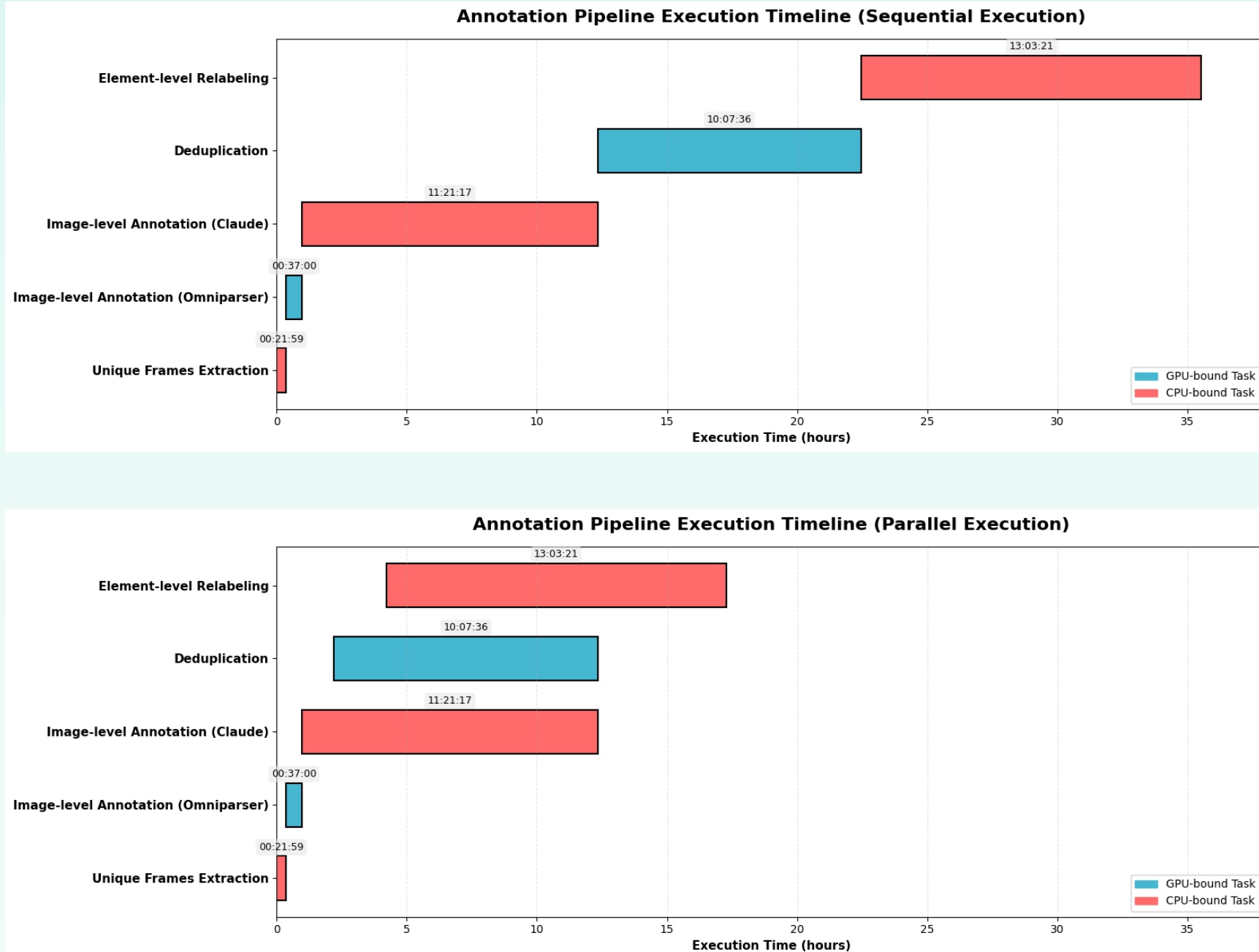
We did a human evaluation on randomly sampled 500 elements, which is show on the first figure on the right. We comparing the labels' quality generated by Gemini and Claude.

Scalability:

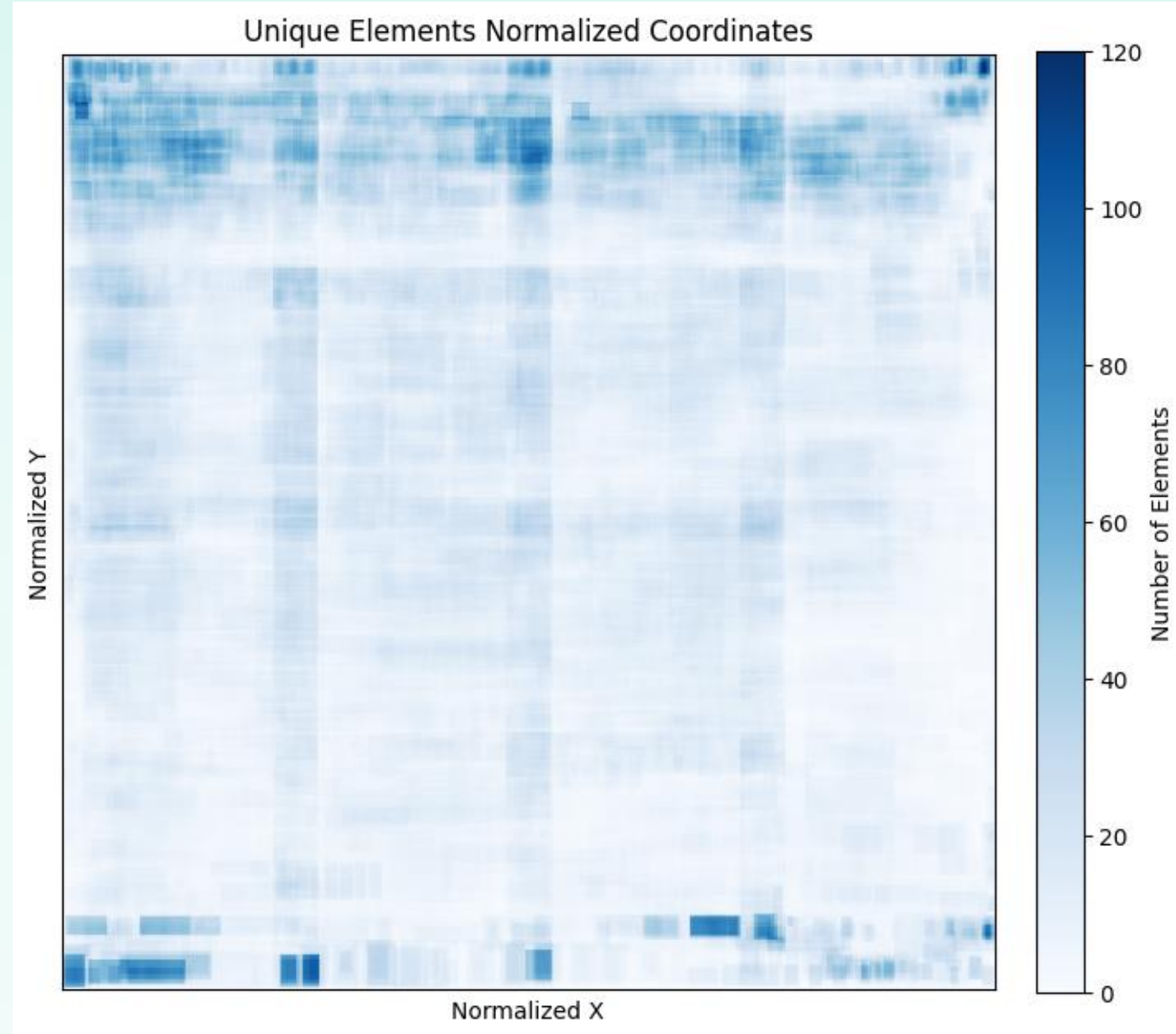
We did a human evaluation on randomly sampled 500 elements, which is show on the first figure on the right. We comparing the labels' quality generated by Gemini and Claude.



We randomly sampled 500 elements and manually checked the generated **bounding-box coordinates** and **labels** by both Gemini and Claude. We can see that Gemini generates marginally more number of correct labels.



Our annotation pipeline has been implemented with parallel execution that reduces the total execution time by more than half.



A visualization of normalized coordinates of the collected unique UI elements. We can see that most UI elements are located either on the top or the bottom of the screen.